

Análise e Projeto de Software Orientado a Objetos

Diagrama de Classes de Design

Leonardo S. Victorio

Faculdade de Computação - UFMS

2026/1



Sumário

- 1 Design da Camada de Domínio
- 2 Responsabilidades dos Objetos
- 3 Visibilidade entre Objetos
- 4 Modelagem Dinâmica
- 5 Pós-condições Condicionais e sobre Coleções
- 6 Consultas de Sistema
- 7 Delegação e Acoplamento Baixo
- 8 Diagrama de Classes de Design
- 9 Conclusão



De onde Viemos — O que Vem a Seguir

Após a **análise** (casos de uso, modelo conceitual, contratos), inicia-se o **projeto de software**. O projeto se divide em:

Projeto Lógico

- Diagramas de classes evoluídos a partir do modelo conceitual.
- Diagramas de interação (comunicação ou sequência).
- Resultado: o **DCD**.

Projeto Tecnológico

- Camada de interface (UI).
- Camada de persistência.
- Segurança, comunicação, etc.

Foco desta aula

Projeto lógico, também chamado de **design da camada de domínio**. Cobre a **modelagem dinâmica** e a construção do **Diagrama de Classes de Design (DCD)**.

O que o Projetista Precisa Ter em Mãos

O projeto lógico pode ser realizado de forma **sistemática**, desde que existam dois artefatos da análise:

1. Modelo Conceitual

Classes, atributos e associações que representam a estrutura da informação do domínio.

2. Contratos de Op./Consultas

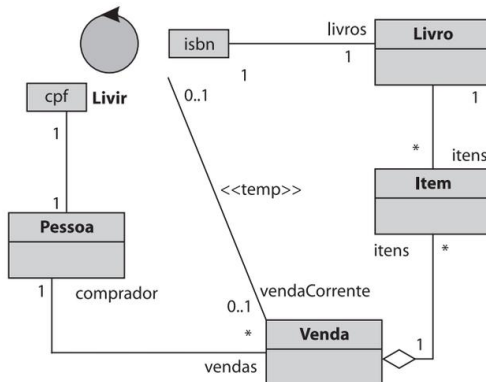
Pré- e pós-condições que especificam **o que** deve acontecer em cada operação de sistema.

Processo do projeto lógico

- 1 Construir um **diagrama de interação** para cada operação e consulta de sistema.
- 2 Usar esses diagramas para decidir **quais métodos** ficam em cada classe.
- 3 Construir e refinar progressivamente o **DCD**.

Modelo Conceitual de Referência (Figura 9.13)

O sistema *Livir* (livraria) será usado nos exemplos da aula. Este é o ponto de partida do DCD.



Classes: *Livir* (controlador), *Pessoa*, *Venda*, *Item*, *Livro*. A associação **<<temp>>** *vendaCorrente* guarda a venda em andamento.

Três Formas de Modelagem Dinâmica

Forma	Vantagem	Desvantagem
Algoritmos	Fáceis de escrever	Ocultam as conexões entre objetos; induzem alto acoplamento
Diag. Comunicação	Linhas de visibilidade explícitas; mostram bem a distribuição de responsabilidades	Difíceis de organizar em colaborações complexas
Diag. Sequência	Mais fáceis de ler	Não explicitam visibilidade; permitem comunicações inválidas

Por que usar diagramas, e não só algoritmos?

Sem método sistemático, as responsabilidades tendem a se **concentrar** em uma ou duas classes — gerando código tão desestruturado quanto *programas espagete*. Os diagramas tornam visíveis as ligações e ajudam a distribuir bem as responsabilidades.

O Erro Comum: “Simulação do Mundo Real”

Princípio

Um sistema OO representa as **informações** sobre o mundo real, **não** as coisas em si.

- Métodos **não** correspondem a ações do mundo real.
- Eles realizam internamente os contratos de operações de sistema.
- Por isso, métodos só aparecem na atividade de **projeto**, nunca na análise.

Consequência do erro

Sem técnica, classes como *Venda*, *Livro* e *Pagamento* acabam sem métodos relevantes, enquanto a controladora (*Livir*) ou classes como *Comprador* fazem tudo. **Isso é estrutura concentradora**, não OO de verdade. Seria preferível um bom projeto estruturado a um OO assim.

Classificação das Responsabilidades — Tabela 9.1

Para distribuir bem os métodos entre as classes, o livro classifica as responsabilidades em **dois grupos**, cada um com **três subgrupos**:

	Conhecer (consultas)	Fazer (comandos)
Sobre si mesmo (atributos)	<code>getAtributo()</code>	<code>setAtributo(v)</code>
Sobre vizinhanças (associações)	<code>getPapel()</code>	<code>addPapel(o)</code> <code>removePapel(o)</code>
Outros (derivado / coordenado)	<code>getDerivado()</code> <code>consultaXYZ()</code>	métodos delegados (nomes variam)

Conhecer corresponde a **consultas** (devolvem valor sem alterar estado).
Fazer corresponde a **operações** (alteram o estado de algum objeto).



Responsabilidades de Conhecer — Detalhes

Sobre si mesmo

Acesso ao valor dos atributos.
Prefixo **get** + nome do atributo.

Ex.: se *Pessoa* tem *dataNascimento*,
o método é *getDataNascimento()*.

Sobre vizinhanças

Acesso aos objetos associados.
Prefixo **get** + nome do **papel**.

Ex.: *Pessoa* associada a *Reserva* pelo
papel **reservas** dá *getReservas()*.

De forma derivada

Informação **calculada** a partir de
outras.

Mesmo prefixo **get**.

Ex.: *getTotal()* em *Venda* retorna
a soma dos preços dos itens.

Normalmente corresponde a um
atributo derivado (marcado
/total).

Os três subgrupos usam o mesmo
prefixo **get**. A diferença está em **de**
onde vem a informação.

Responsabilidades de **Fazer** — Detalhes

Sobre si mesmo

Alteração do valor de atributos.
Prefixo **set** + nome do atributo.

Ex.: `setDataNascimento(umaData)`
em *Pessoa*.

Sobre vizinhanças

Adição e remoção de associações.
Prefixos **add** e **remove** + nome do papel.

Ex.: `addReservas(r)` e
`removeReservas(r)` em *Pessoa*.

De forma coordenada

Operações múltiplas, coordenadas pelo objeto com melhor visibilidade para os alvos. São chamadas **métodos delegados**.

Ex.: marcar todos os livros de uma venda como entregues — quem coordena é a *Venda*, pois ela enxerga seus itens.

Nomes dos delegados

Não há prefixo padrão. O nome não deve incluir o nome da classe.

Ex.: `encerrar()`, não
`encerrarVenda()`.

Operações Básicas são Implícitas no DCD

Cada classe **já tem por padrão** todos os métodos básicos:

Operações implícitas em qualquer classe

- `create`, `destroy`
- Para cada atributo: `getAtrib()` e `setAtrib(v)`
- Para cada associação: `getPapel()`, `addPapel(o)`, `removePapel(o)`

Conta da padaria

Uma classe com **6 associações** e **15 atributos** teria, só de básicas: 1 `create` + 1 `destroy` + 15 `set` + 15 `get` + 6 `add` + 6 `remove` + 6 `get` = **50 métodos**.

No DCD só aparecem explicitamente os **métodos delegados** (e atributos derivados). Os básicos ficam subentendidos pela existência dos atributos e associações.



Por que Visibilidade Importa?

Para que dois objetos possam **trocar mensagens**, deve existir **visibilidade** entre eles. Sem visibilidade, não há comunicação.

Quatro formas básicas de visibilidade

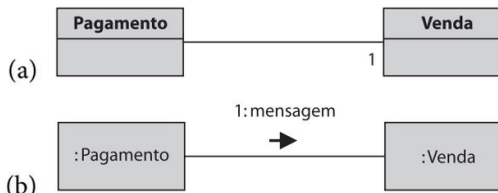
- ❶ **Por associação** — existe ligação definida no modelo conceitual.
- ❷ **Por parâmetro** — o objeto chega como argumento de um método.
- ❸ **Localmente declarada** — o objeto chega como retorno de uma consulta.
- ❹ **Global** — o objeto é declarado globalmente (Singleton).

Assimetria

Nenhuma forma é **simétrica**: se x enxerga y , isso **não** implica que y enxerga x .



Visibilidade por Associação — Para Um (Figura 9.1)



Multiplicidade 1

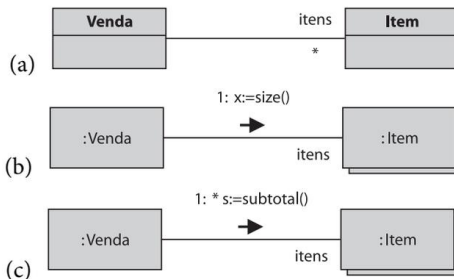
Quando a multiplicidade no lado-alvo é **1**, cada instância origem enxerga **uma** instância alvo **diretamente**.

No exemplo: cada **:Pagamento** pode enviar mensagens diretamente à **:Venda** associada.

Pré-condições restringem

Se o contrato exige $\text{pagamento} \rightarrow \text{size}() = 1$, uma associação $0..1$ passa a valer como **para um** naquela operação.

Visibilidade por Associação — Para Muitos (Figura 9.2)



Multiplicidade $*$, $1..*$, $0..1$

A visibilidade é para uma **coleção** de instâncias.

Duas formas de mensagem

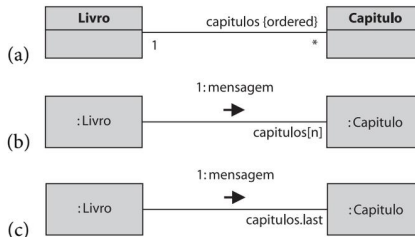
(b) Para o conjunto:

1: `x:=size()`
operação na estrutura.

(c) Para cada elemento:

1: `*s:=subtotal()`
o `*` antes da mensagem indica envio a **cada** elemento.

Associações Ordenadas e Qualificadas (Figuras 9.3 e 9.4)

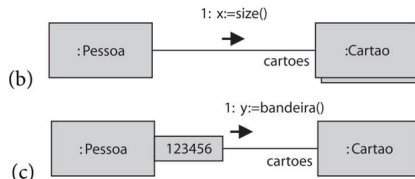


Ordenadas {ordered}

Acesso por **posição**:

`capitulos[n]` (n-ésimo)

`capitulos.last` (último)

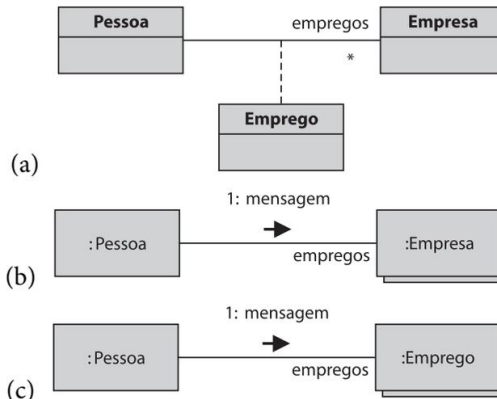


Qualificadas

Com a **chave do qualificador**, acesso direto ao elemento: `cartoes[123456]` dá visibilidade imediata para aquele cartão.



Classe de Associação (Figura 9.6)



Dupla visibilidade

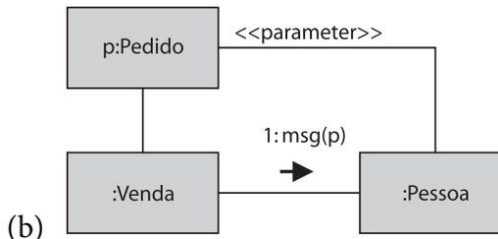
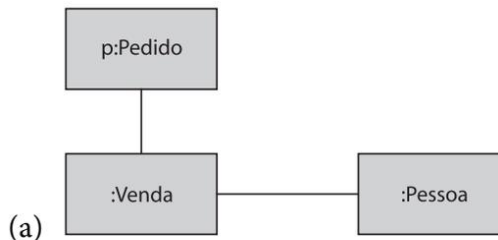
Pelo papel **empregos**, uma **:Pessoa** enxerga **dois** conjuntos:

- o conjunto de *Empresa*;
- o conjunto de *Emprego*.

Volta sempre para um

Uma instância de *Emprego* enxerga **exatamente uma** *Pessoa* e **exatamente uma** *Empresa*.

Visibilidade por Parâmetro (Figura 9.10)



Como funciona

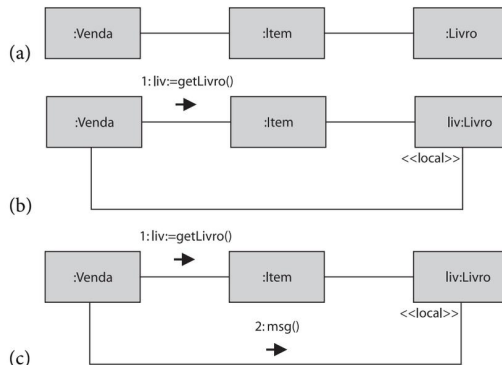
Ao executar um método, um objeto **recebe outro como parâmetro** e passa a poder enviar mensagens a ele — **mesmo sem associação** entre suas classes.

Validade temporária

A visibilidade por parâmetro existe **só durante** a execução do método. Depois desaparece, igual a uma variável local.



Visibilidade Localmente Declarada (Figura 9.11)



Como funciona

Quando um objeto recebe outro como **retorno** de uma consulta, passa a ter visibilidade **local** sobre ele.

No exemplo, `:Venda` chama `getLivro()` em `:Item` e obtém `liv:Livro`.

Também é temporária

Vale apenas dentro do método.

Princípio “Não Fale com Estranhos”

Regra (Lei de Demeter)

Evite ao máximo usar visibilidade por parâmetro ou local para *enviar mensagens* a objetos.

- Mensagens devem trafegar pelas **linhas de visibilidade por associação**.
- Objetos recebidos como parâmetro/retorno deveriam apenas ser **repassados**, não chamados.
- Exceção aceitável da local: quando o método que retorna o objeto *cria* esse objeto.

Por que?

Cada mensagem fora das associações cria **acoplamento extra** — não previsto no modelo conceitual — e degrada o projeto.

Visibilidade Global (Figura 9.12)



Padrão Singleton

Visibilidade global é válida **apenas** para instância **única** de uma classe — o padrão *Singleton*.

Costuma ser usada para classes de **serviço**: *ConversorDeMoedas*, *ContadorDeTempo*.

Entidades do domínio (*Venda*, *Livro*) **nunca** devem ser Singletons.

Pós-condições e Diagramas de Interação

Os contratos dizem **o que** deve acontecer (pós-condições).

Os diagramas mostram **como** os objetos trocam mensagens para fazê-lo.

Quatro princípios

- 1 A visibilidade entre objetos é regida pela **multiplicidade** (e por pré-condições).
- 2 Cada pós-condição vira o envio de uma **mensagem básica** ao responsável imediato.
- 3 O fluxo **sempre inicia** no **controlador** recebendo mensagem da interface.
- 4 Sem visibilidade para o alvo, o controlador deve **delegar** ao objeto que enxerga.

Tipo	Posição	Quantidade
Operação de sistema	Raiz	1 por diagrama
Mensagens básicas	Folhas	1 por pós-condição
Mensagens delegadas	Ramos	quando preciso



Quem deve criar a instância?

Prefira (nesta ordem):

- 1 Classe que tenha **composição/agregação** com a classe criada.
- 2 Classe que tenha **um-para-muitos** com ela.
- 3 Classe que possua os **dados de inicialização** e esteja associada à classe.

Cada regra só se aplica se a anterior não for possível.

Exemplo *Livir*: criar um *Item*

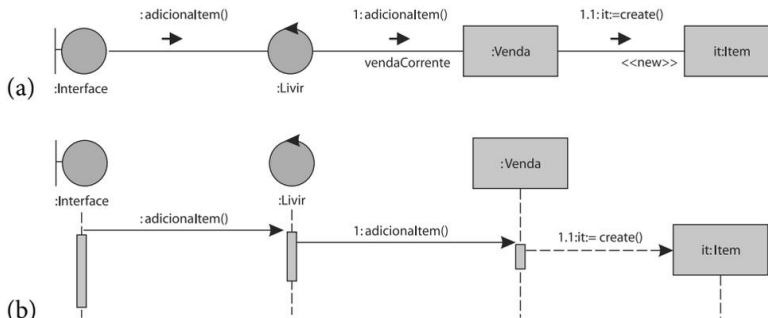
Quem cria? **Venda** — pois há agregação *Venda-Item*.

Livro, *Pessoa* e *Livir* **não** criariam:

- *Livir* sequer tem associação com *Item*.
- *Livro* associa-se a *Item*, mas *Venda* ganha pela agregação.

Criação de Instância — Diagramas (Figura 9.14)

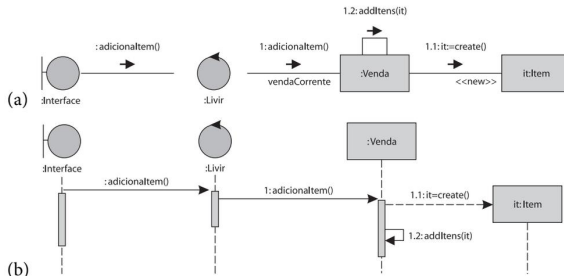
Contrato: `Livir::adicionaItem(...)` cria um novo *Item*.



Livir não tem visibilidade para *Item*, então delega (mensagem 1) à `vendaCorrente:Venda`, que então executa o `create` (mensagem 1.1) — a folha da árvore.

Criação de Associação (Figura 9.15)

Toda instância recém-criada precisa ser **imediatamente ligada** a outra para se tornar acessível.



Pós-condição

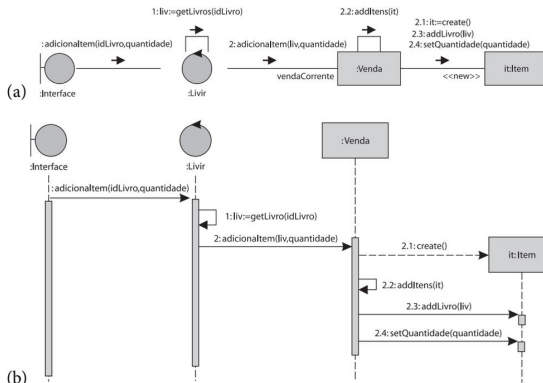
`vendaCorrente.addItens(item)`

Duas formas simétricas

`venda.addItens(item)` e `item.addVenda(venda)` produzem o mesmo efeito. Prefira o receptor **mais próximo do controlador**.

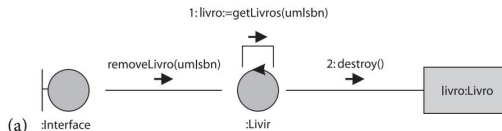
Caso Completo — Cria, Associa e Atualiza (Figura 9.17)

Contrato: `Livir::adicionaItem(idLivro, quantidade)`



```
1: liv := getLivros(idLivro) (qualif.) 2: adicionaItem(liv, qtd) (delega) 2.1: it := create() 2.2: addItens(it) 2.3: addLivro(liv) 2.4: setQuantidade(qtd)
```

Destruição de Instância (Figura 9.18)



destroy()

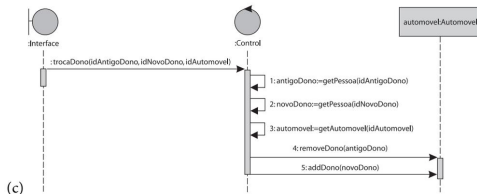
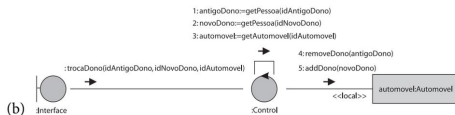
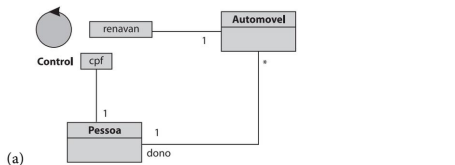
Após **destroy**, **nenhuma mensagem** pode mais ser enviada ao objeto.

Vale o mesmo padrão *Criador*: composição, agregação ou associação 1-para-*.

Sem órfãos

Com contratos bem formados, nenhum objeto fica com referência pendente para o destruído.

Remoção de Associação (Figura 9.19) — Troca de Dono



Pós-condições

`removeDono(antigo)`
AND
`addDono(novo)`

Controladora obtém os
três objetos via consultas
qualificadas e envia
remove+add.

Algumas pós-condições só valem se uma condição inicial for satisfeita. Nos diagramas, isso vira uma **mensagem com guarda**.

Exemplo do livro: desconto de 10% se total > 1000

```
Context Livir::aplicaDesconto()  
pre: vendaCorrente.size() = 1  
post: vendaCorrente.valorTotal > 1000 IMPLIES  
      vendaCorrente.setValorTotal(.../1.1)
```

No diagrama

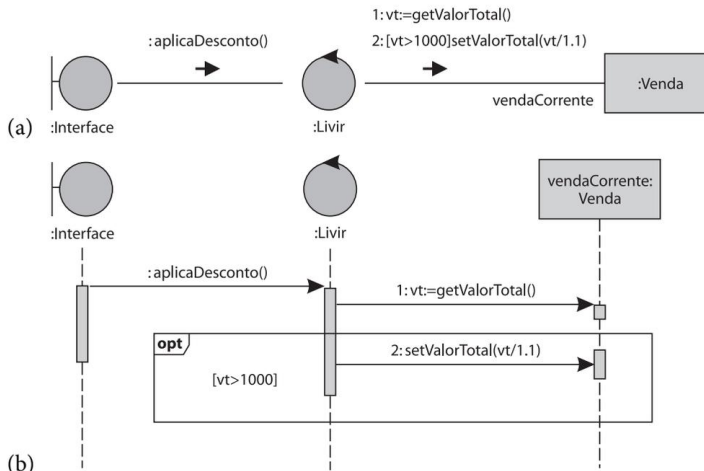
Comunicação: a guarda vai entre colchetes antes da mensagem:

2: [vt>1000]: setValorTotal(vt/1.1)

Sequência: a mensagem fica dentro de um bloco **opt** com a condição.



Mensagem Condicional nos Diagramas (Figura 9.20)



Se $vt \leq 1000$, a mensagem simplesmente **não é executada**.
Para um *if-then-else*, basta uma segunda mensagem com a guarda negada.
Para um *case*, uma mensagem por caso.

Pós-condições sobre Coleções — Iteração

Quando o contrato atualiza **todos os elementos** de uma coleção, usa-se o prefixo ***** antes da mensagem.

Exemplo do livro: aumentar 10% em todos os livros

```
Context Livir::aumentaPrecos()  
post: livros->forAll(livro |  
    livro.setPreco(livro.preco@pre * 1.1))
```

Todos os elementos

```
1: *preco := getPreco()  
2: *setPreco(preco*1.1)
```

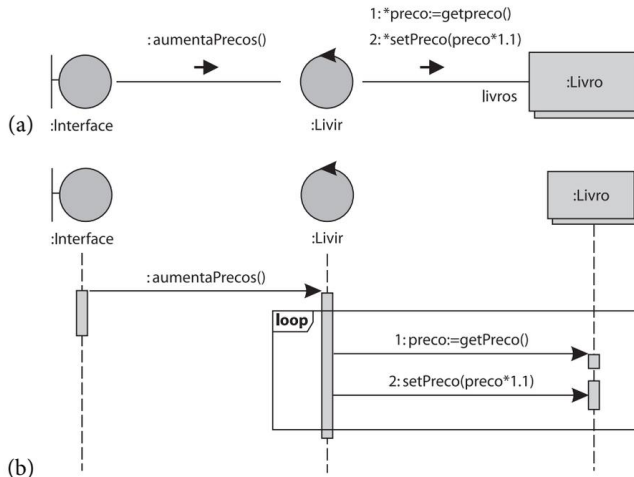
Apenas alguns (filtro)

```
2: *[preco<100] setPreco(...)  
Só envia a mensagem aos elementos  
que satisfazem a condição.
```

No diagrama de sequência

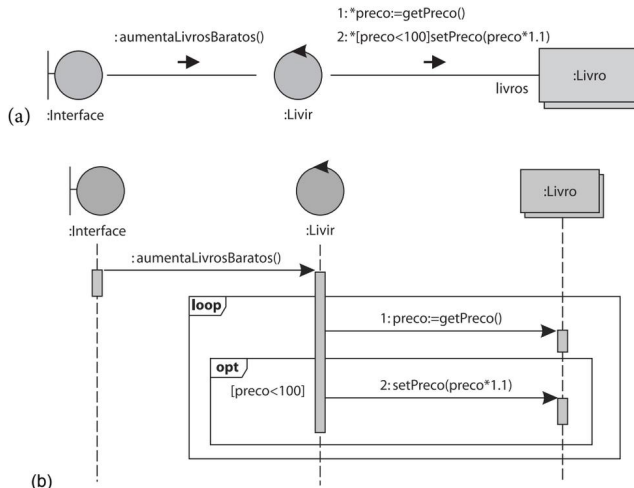
O ***** vira um bloco **loop**; o ***[cond]** vira **loop** com **opt** interno.

Iteração nos Diagramas (Figura 9.21)



Livir envia duas mensagens **a cada** elemento de livros: `getPreco()` e `setPreco(preco*1.1)`. O retângulo empilhado representa a **coleção**.

Iteração com Filtro (Figura 9.22)



`Livir::aumentaLivrosBaratos()` — só aumenta os livros com `preco < 100`. No diagrama de sequência, dentro do `loop` aparece um `opt [preco<100]` que filtra.

Consultas de Sistema — Três Tipos de Mensagem

Em diagramas de consulta, as mensagens se organizam em três níveis:

1. Consulta de sistema (raiz)

Vem da interface, é a única mensagem “de entrada” no diagrama. Retorna **alfanumérico** ou uma estrutura DTO (*Data Transfer Object*).

2. Consultas básicas (folhas)

São os **get** de atributos e papéis de associação. Não precisam ser detalhadas.

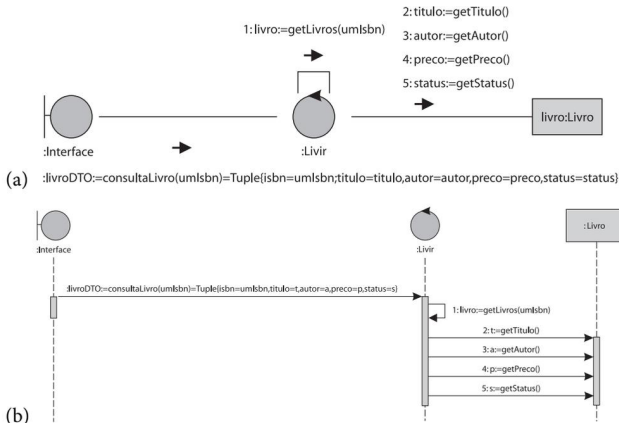
3. Consultas intermediárias (ramos)

Correspondem a atributos derivados ou a consultas com cálculo — e ficam nas classes mais próximas dos dados.



Exemplo CRUD — Consulta de Livro (Figura 9.23)

Contrato: `consultaLivro(umIsbn)` retorna um DTO com os dados do livro.



Livir usa `getLivros(umIsbn)` (qualificada) para obter o `livro`, depois envia `getTitulo`, `getAutor`, `getPreco` e `getStatus` (todas básicas). A `Tuple{...}` é montada na chamada raiz.

Quando não Vale a Pena o Diagrama

Limitação dos diagramas

Diagrama de comunicação e sequência **não representam bem** funções matemáticas, filtros ou combinações complexas de valores.

Recomendação do livro

Para consultas com lógica complexa, use **algoritmo** ou programação direta a partir do contrato, mantendo em mente:

- Se a consulta retorna alfanumérico simples, considere transformá-la em **atributo derivado**.
- Procure agrupar consultas básicas em estruturas reusáveis (DTO já filtrado, tupla, etc.).



Padrões de Consulta com Filtro

Suponha o modelo da Figura 9.24: *Livir* tem catálogo de *Livro*, e queremos filtrar por autor, ano ou gênero.

Padrão	Como funciona	Quando usar
A	Retorna tudo ; quem chamou filtra.	Muitos filtros, poucas chamadas; mas gera código repetido.
B	Um método por filtro : getPorAutor(a), getPorGenero(g)...	Poucos filtros, muitas chamadas.
C	Objeto filtro como parâmetro: getCatalogo(f:Filtro)	Muitos filtros possíveis; chamadas simples.

Escolha do padrão

Depende da **quantidade de filtros** e da **quantidade de chamadas**. Não há padrão sempre correto.

Padrão C — Objeto Filtro (Figura 9.25)

FiltroLivros

+autorNome : String
+periodo : Intervalo
+genero : String

Como se usa

```
f := FiltroLivros.newInstance()  
f.setAutorNome("raul")  
f.setPeriodo(Intervalo[2004..2006])  
livros := self.getCatalogo(f)
```

Como funciona

Para cada livro do catálogo, o método verifica se ele bate com cada campo *não nulo* do filtro. Atributos nulos do filtro **não restringem**.

FiltroLivros **não é classe conceitual** — é uma classe de **projeto** introduzida apenas no DCD.

Duas Abordagens Opostas: Concentração vs. Delegação

Quando o controlador **não enxerga** o objeto que executaria a operação, há dois caminhos possíveis:

Concentração

O controlador **obtém uma referência** ao alvo (via getter) e executa a operação **ele mesmo**.

Analogia (livro): João pede o telefone do buffet a Pedro e liga ele próprio.

⇒ Cria conexão extra; pouco reuso.

Delegação

O controlador **passa a responsabilidade** ao vizinho que enxerga o alvo.

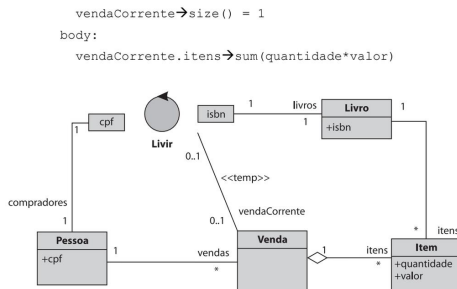
Analogia (livro): João passa os parâmetros a Pedro e nunca toma conhecimento do telefone.

⇒ Menos conexões; gera métodos reusáveis.

Princípio

Evite conexões entre objetos que não estavam ligados no modelo conceitual. Alto acoplamento = projeto ruim.

Modelo do Exemplo (Figura 9.26)



Cenário

Consulta: **valor total da venda corrente.**

Cálculo: somar $\text{quantidade} \times \text{valor}$ de cada *Item* da venda.

Livir não tem associação com *Item*. Só com *Venda* (via *vendaCorrente*).

Pseudocódigo concentrador

CLASSE *Livir*

MÉTODO getTotalVendaCorrente() : Moeda

 itens := vendaCorrente.getItems()

 total := 0

 PARA CADA item EM itens FAÇA

 total := total + (item.getValor() * item.getQuantidade())

 FIM PARA

 RETORNA total

Avaliação

- Está correto. Resolve o problema.
- Mas **não gera nada reusável**: o cálculo fica trancado dentro de `getTotalVendaCorrente`.
- *Livir* passa a depender da estrutura interna de *Item* (`valor`, `quantidade`).

Versão com Delegação — Cada Classe Faz o Seu

Cada cálculo vira um **atributo derivado** na classe que tem os dados:

Definições OCL

```
Context Item::subtotal:Moeda
  derive: quantidade*valor
Context Venda::valorTotal:Moeda
  derive: itens->sum(subtotal)
```

Pseudocódigo distribuído

```
CLASSE Livir
  MÉTODO getTotalVendaCorrente():Moeda
    RETORNA vendaCorrente.getValorTotal()

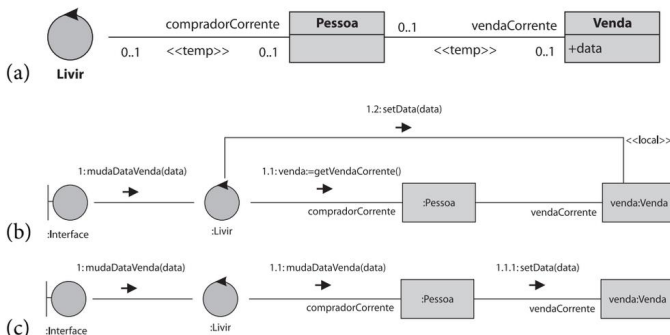
CLASSE Venda // soma os subtotais dos itens
CLASSE Item  // calcula quantidade*valor
```

Resultado

`getSubtotal()` e `getValorTotal()` viram métodos **reusáveis** em qualquer contexto. *Livir* não conhece mais *Item*.

Delegação em Comandos (Figura 9.27)

Operação: `mudaData(umaData)` — atualiza a data da venda corrente.



(b) Concentrador

Livir obtém `venda` via `getVendaCorrente()` e chama `setData` diretamente. Acopla-se ao atributo interno.

(c) Delegado

Livir envia `mudaDataVenda(data)` à `vendaCorrente`, que internamente faz `setData`. Livir nunca conhece `data`.

O que é o DCD?

Definição

O **Diagrama de Classes de Design (DCD)** é o produto central do projeto lógico. Mostra as classes da camada de domínio com tudo que é preciso para **implementá-las**.

Ponto de partida

A **primeira versão do DCD é uma cópia** do modelo conceitual. Em seguida, ele é progressivamente modificado conforme os diagramas de interação são construídos.

Quando o DCD está pronto?

Quando todos os contratos de operação e consulta de sistema foram modelados, e o diagrama tem informação suficiente para que cada classe seja implementada.

Cinco Modificações Feitas no DCD durante o Projeto

- ➊ **Adição de métodos.** Métodos **delegados** descobertos nos diagramas de interação são adicionados às classes correspondentes.
- ➋ **Direção das associações.** Antes não-direcionais; passam a ter setas conforme o sentido das mensagens trocadas nos diagramas.
- ➌ **Detalhamento de tipos.** Tipos abstratos podem virar concretos (*lista* → *array* ou *LinkedList*). Tipos de atributos podem ser fixados.
- ➍ **Reestruturação.** Novas classes de projeto podem ser introduzidas (estratégias, serviços, filtros). O DCD pode **divergir** do conceitual.
- ➎ **Visibilidade de atributos.** No conceitual todos são públicos. No DCD podem aparecer **privados** ou **protegidos** para encapsular estado interno.



O que NÃO se Coloca no DCD — Métodos Básicos

Toda classe **já tem por padrão** os métodos básicos — eles podem ser deduzidos da estrutura:

Métodos implícitos em qualquer classe

- `create`, `destroy`
- Para cada **atributo**: `getAtrib()`, `setAtrib(v)`
- Para cada **associação**: `getPapel()`, `addPapel(o)`, `removePapel(o)`

Conta da padaria

Uma classe com **6 associações e 15 atributos** teria **50 métodos básicos**.
Listar todos polui o diagrama.

O que entra explicitamente no DCD

Apenas os **métodos delegados** (e atributos derivados não triviais), já que não podem ser deduzidos da estrutura.

Método Delegado Gera Método no DCD (Figura 9.28)



Como ler

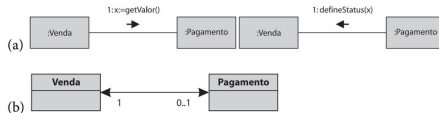
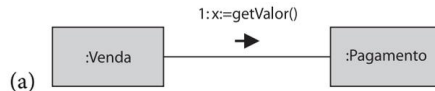
- (a) A instância `:Venda` recebe a mensagem delegada `atualizaImpostos(aliquota)`.
- (b) Consequência no DCD: a classe *Venda* deve **declarar** esse método.

Regra geral

Sempre que um objeto recebe uma mensagem **delegada** num diagrama de interação, a classe correspondente ganha esse método no DCD.

Direção das Associações no DCD (Figuras 9.29 e 9.30)

A direção de cada associação no DCD vem do **sentido das mensagens** nos diagramas de interação.



Unidirecional (9.29)

Só *Venda* envia mensagens a *Pagamento*
⇒ seta de Venda → Pagamento.

Bidirecional (9.30)

Mensagens em **ambos os sentidos**
⇒ setas nas duas pontas.

DCD vs. Modelo Conceitual

Aspecto	Modelo Conceitual	DCD
Propósito	Representar o domínio	Guiar a implementação
Atributos	Todos públicos	Podem ser privados/protegidos
Métodos	Só op./consultas de sistema (na controladora)	Métodos delegados explicitados
Associações	Não direcionais	Direcionadas pelo fluxo
Novas classes	Não	Sim (estratégias, filtros, serviços)
Tipos	Abstratos	Podem ser concretos (<i>array</i> , etc.)

O DCD é vivo

Vai sendo refinado a cada contrato analisado. Mudanças eventuais no modelo conceitual devem ser propagadas ao DCD.

- O **design da camada de domínio** consiste em modelagem dinâmica e construção do **DCD**.
- Responsabilidades dos objetos: **conhecer** e **atualizar**, cada uma com três subgrupos.
- Quatro formas de visibilidade. Apenas a **por associação** é permanente.
- O **Padrão Criador** define quem cria instâncias — prioriza composição e agregação.
- **Delegação** produz código com menor acoplamento e maior coesão do que concentração.
- O **DCD** evolui do modelo conceitual: métodos adicionados, associações direcionadas, tipos detalhados.
- “**Não Fale com Estranhos**” restringe a comunicação entre objetos sem associação semântica.
- Pós-condições condicionais usam **mensagens guardadas**; iterações usam o prefixo *****.



Resumo do Processo de Design Lógico

- ❶ Partir do **modelo conceitual** (primeira versão do DCD).
- ❷ Para cada contrato de **operação de sistema**:
 - a. Identificar as pós-condições e as visibilidades disponíveis.
 - b. Construir o **diagrama de comunicação** iniciando no controlador.
 - c. Delegar responsabilidades respeitando as linhas de visibilidade.
- ❸ A cada mensagem recebida por um objeto, **adicionar o método** à sua classe no DCD.
- ❹ **Direcionar as associações** conforme o fluxo de mensagens.
- ❺ Identificar **consultas intermediárias** e transformá-las em atributos derivados quando possível.
- ❻ Repetir até todos os contratos estarem modelados.
- ❼ Revisar o DCD garantindo **baixo acoplamento** e **alta coesão**.



Conexão com o Restante do Projeto

Com o DCD estabilizado, o projeto tecnológico pode avançar:

Camada de Interface

Mantida **desacoplada** da camada de domínio. O controlador-fachada é o único ponto de contato. A interface opera apenas com dados alfanuméricos.

Camada de Persistência

Pode ser projetada de forma **automática** a partir do DCD, sem que o designer precise se preocupar com detalhes de armazenamento em memória secundária.

O DCD é vivo

À medida que novos contratos são analisados e a implementação avança, o DCD é refinado. Mudanças no modelo conceitual devem ser propagadas.

Referências Bibliográficas

-  WAZLAWICK, Raul Sidnei. *Análise e Projeto de Sistemas de Informação Orientados a Objetos*. 3. ed. Rio de Janeiro: Elsevier, 2015. Cap. 9 — Projeto da Camada de Domínio.
-  LARMAN, Craig. *Utilizando UML e Padrões*. 3. ed. Porto Alegre: Bookman, 2007. Cap. 9 — Domain Layer Design.
-  GAMMA, Erich et al. *Padrões de Projeto: soluções reutilizáveis de software OO*. Porto Alegre: Bookman, 2000.
-  LES/PUC-Rio. *UML: Diagrama de Classes*. Laboratório de Engenharia de Software, PUC-Rio.

Dúvidas e Discussões

Leonardo S. Victorio

leonardo.victorio@ufms.br

Faculdade de Computação — UFMS

“Fazer design de software orientado a objetos deve ser entendido como um método preciso, guiado por padrões aprendidos, e não simplesmente como o ato de criar classes e associar métodos ad hoc a elas.”
— Wazlawick (2015)

